UNIT- IV

Planning

We have defined AI as the study of rational action, which means that planning—devising a plan of action to achieve one's goals—is a critical part of AI. We have seen two examples of planning agents so far:

1. the search-based problem-solving agent and
2. the hybrid logical agent.

In this chapter we introduce a representation for planning problems that scales up to problems that could not be handled by those earlier approaches.

DEFINITION OF CLASSICAL PLANNING:

Classical planning refers to the process of building complex plan of actions by the agent while utilizing the structure of a problem efficiently.

The problem-solving agent can find sequences of actions that result in a goal state. But it deals with atomic representations of states and thus needs good domain-specific heuristics to perform well.

The hybrid propositional logical agent can find plans without domain-specific heuristics because it uses domain-independent heuristics based on the logical structure of the problem. But it depends on ground (variable-free) propositional inference, which means that it may be filled when there are many actions and states.

For example, in the wumpus world, the simple action of moving a step forward had to be repeated for all four agent orientations, T time steps, and n^2 current locations.

In response to this, planning researchers have settled on a factored representation— one in which a state of the world is represented by a

collection of variables. We use a language called PDDL, the Planning Domain Definition Language, that allows us to express all PDDL actions with one action schema. There have been several versions of PDDL; we selected a simple version. To define a search problem, PDDL describes the four things:

- the initial state,
- the actions that are available in a state,
- the result of applying an action, and
- the goal test.

States are represented using a set of changing aspects of the world(fluents). They are integrated using conjunction i.e., 'Λ'.

Actions are described by a set of action schemas that implicitly define the ACTIONS(s) and RESULT (s, a) functions needed to do a problem-solving search.

Classical planning usually focuses on the problems which do not modify most of the things with its actions. The set of actions should be represented in a concise form like 'Δ' instead of specifying the overall objects. Another way of concise representation used by PDDL is the specification of only the things which are changed. The remaining things are simply ignored.

Another way of concise representation for actions which are variable-free is to use a single action schema to represent a set of actions. A typical schema carries action name, variables involved, a precondition and the effect. Here, the precondition represents the states of executing the action and effect represents the output of action. Both these fields are conjunctions of literals. If a state satisfies the precondition, then the action associated with it is called 'applicable'.

Example: The spare tire problem

The problem of spare tire occurs when a tire is punctured and that tire must be replaced with the spare tire (i.e., new tire) to make the vehicle free from puncture.

Initial State of the Vehicle: (Flat → Axle) ^ (Spare → Trunk)

according to the initial state, a flat tire (i.e., puncture tire) is on the axle of the vehicle and spare tire is on the trunk.

Goal State of the Vehicle: (Spare → Axle) ^ (Flat → Trunk)

In the goal state, the spare tire must be mounted on the axle of the vehicle and flat tire must be mounted on the trunk of the vehicle.

The following actions must be performed to replace a flat tire with a spare tire on the axle.

1) First, separate the flat tire from the axle.
2) Now, separate the spare tire from the trunk.
3) Then place spare tire on the axle
4) Finally, place a flat tire on the trunk.

The following are the sequence of events which must be followed with the above actions to achieve the goal state.

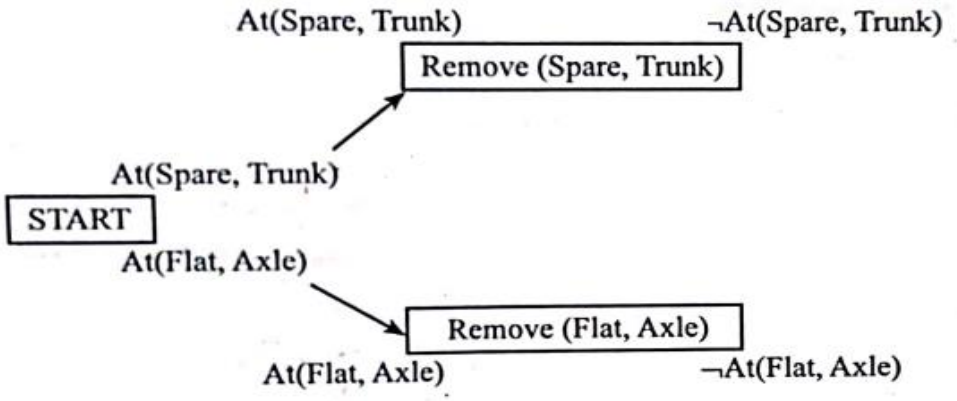
1. Action (Remove (Spare, Trunk))
   Precond: At (Spare, Trunk)
   Effect: ¬At (Spare, Trunk)

This action is performed for removing a spare tire from the trunk with a precondition that the spare tire must be available on the trunk so that it can be removed from the trunk. However, this action gets effected when there is no spare tire on the trunk.

2. Action (Remove (Flat, Axle))
   Precond: At(Flat, Axle)
   Effect: ¬At(Flat, Axle)

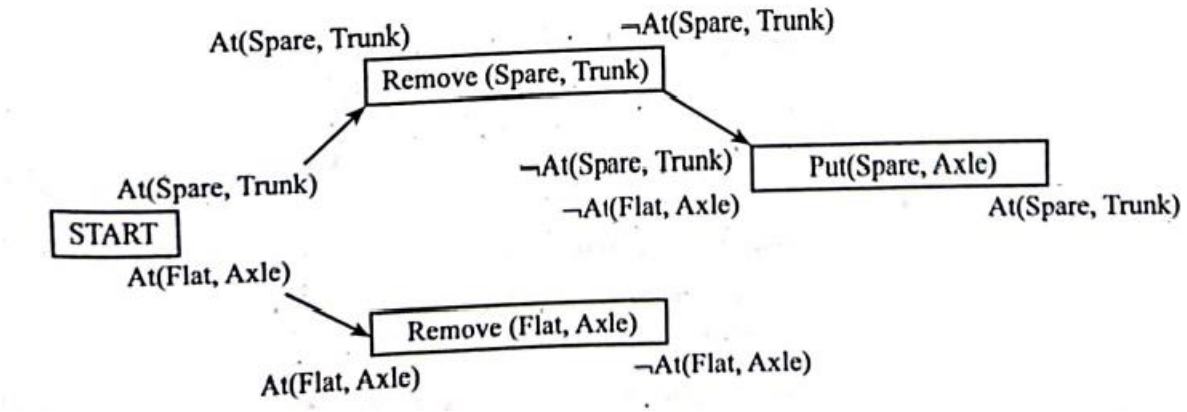
This action is performed for removing flat tire from the axle with a precondition that the flat tire must be available on the axle so that it can be removed from the axle. However, this action gets effected when there is no flat tire on the axle.

in the below figure, actions are represented by the boxes. The left and right sides of the boxes represent preconditions and their effects respectively.



3. Action (Put (Spare, Axle))
   Precond: ¬At (Spare, Trunk) ^ ¬ (Flat, Axle)
   Effect: At (Spare, Trunk)

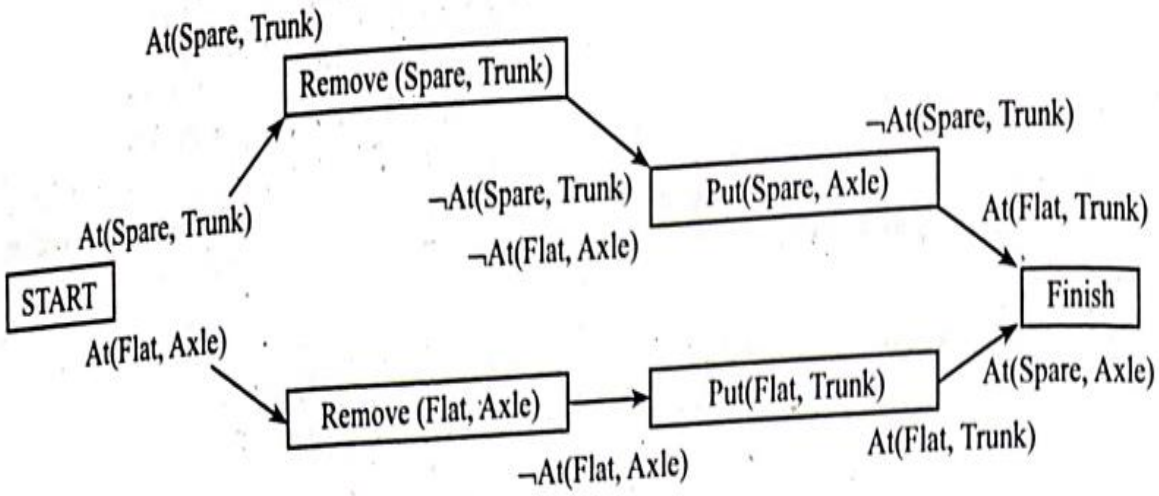
This action is performed by placing spare tire on the axle of the vehicle with a precondition that the spare tire which has to be placed on axle must not be on the trunk (i.e., ¬At (Spare, Trunk)) and flat tire must not be on the axle. However, this action gets effected if the spare tire is still on the trunk.



4. Action(Put(Flat, Trunk)
   Precond: ¬At(Flat,Axle)^ ¬At(Spare, Trunk)
   Effect:At(Flat, Trunk)

This action is performed by placing the flat tire on the trunk with a precondition that the flat tire must not be present on the axle and spare tire must not be on the trunk so that it can mount a new spare tire on the axle of the vehicle. However, this action gets effected when the flat tire is still on the trunk.

The below figure illustrates the goal state of the spare tire problem by following all the actions and their sequence of events.



A solution to the problem is [Remove (Flat, Axle), Remove (Spare, Trunk), PutOn(Spare, Axle) PutOn(Flat, Trunk)].

$$
\begin{array}{l}
Init(Tire(Flat) \wedge Tire(Spare) \wedge At(Flat, Axle) \wedge At(Spare, Trunk)) \\
Goal(At(Spare, Axle)) \\
Action(Remove(obj, loc), \\
\quad \text{PRECOND: } At(obj, loc) \\
\quad \text{EFFECT: } \neg At(obj, loc) \wedge At(obj, Ground)) \\
Action(PutOn(t, Axle), \\
\quad \text{PRECOND: } Tire(t) \wedge At(t, Ground) \wedge \neg At(Flat, Axle) \\
\quad \text{EFFECT: } \neg At(t, Ground) \wedge At(t, Axle)) \\
Action(LeaveOvernight, \\
\quad \text{PRECOND:} \\
\quad \text{EFFECT: } \neg At(Spare, Ground) \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, Trunk) \\
\qquad \wedge \neg At(Flat, Ground) \wedge \neg At(Flat, Axle) \wedge \neg At(Flat, Trunk))
\end{array}
$$

Figure 10.2 The simple spare tire problem.

Example: The blocks world

- The blocks world One of the most famous planning domains is known as the blocks world.
- This domain consists of a set of cube-shaped blocks sitting on a table.
- The blocks can be stacked, but only one block can fit directly on top of another.
- A robot arm can pick up a block and move it to another position, either on the table or on top of another block.
- The arm can pick up only one block at a time, so it cannot pick up a block that has another one on it.
- The **goal** will always be to build one or more stacks of blocks, specified in terms of what blocks are on top of what other blocks.
- For example, a goal might be to get block A on B and block B on C.

$Init(On(A, Table) \land On(B, Table) \land On(C, A)$
$\quad \land Block(A) \land Block(B) \land Block(C) \land Clear(B) \land Clear(C))$
$Goal(On(A, B) \land On(B, C))$
$Action(Move(b, x, y),$
$\quad$ PRECOND: $On(b, x) \land Clear(b) \land Clear(y) \land Block(b) \land Block(y) \land$
$\quad\quad (b \neq x) \land (b \neq y) \land (x \neq y),$
$\quad$ EFFECT: $On(b, y) \land Clear(x) \land \neg On(b, x) \land \neg Clear(y))$
$Action(MoveToTable(b, x),$
$\quad$ PRECOND: $On(b, x) \land Clear(b) \land Block(b) \land (b \neq x),$
$\quad$ EFFECT: $On(b, Table) \land Clear(x) \land \neg On(b, x))$

Figure 10.3 A planning problem in the blocks world: building a three-block tower. One solution is the sequence $[MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)]$.

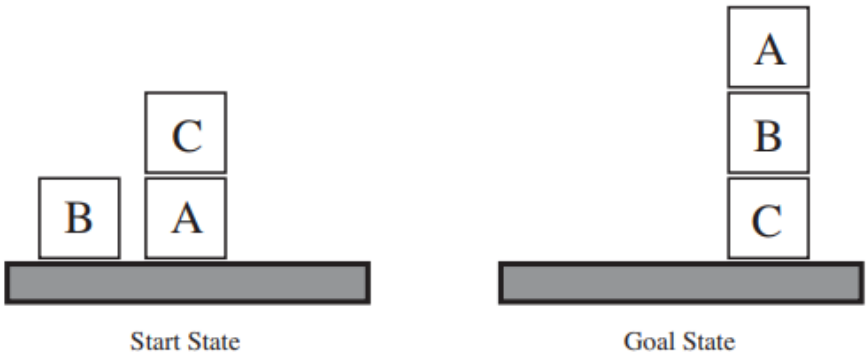


Figure 10.4 Diagram of the blocks-world problem in Figure 10.3.

- We use On (b, x) to indicate that block b is on x, where x is either another block or the table.
- The action for moving block b from the top of x to the top of y will be Move (b, x, y).
- Now, one of the preconditions on moving b is that no other block be on it.
- In first-order logic, this would be ¬∃ x On (x, b) or, alternatively, ∀ x ¬On (x, b).
- Basic PDDL does not allow quantifiers, so instead we introduce a predicate Clear (x) that is true when nothing is on x.
- The action Move moves a block b from x to y if both b and y are clear.

- After the move is made, b is still clear but y is not.
- A first attempt at the Move schema is

  Action(Move(b, x, y),

  PRECOND:On(b, x) ∧ Clear (b) ∧ Clear (y),

  EFFECT:On(b, y) ∧ Clear (x) ∧ ¬On(b, x) ∧ ¬Clear (y)) .
- Unfortunately, this does not maintain Clear properly when x or y is the table.
- When x is the Table, this action has the effect Clear (Table), but the table should not become clear; and when y = Table, it has the precondition Clear (Table), but the table does not have to be clear for us to move a block onto it. To fix this, we do two things.
- First, we introduce another action to move a block b from x to the table:

  Action(MoveToTable(b, x),

  PRECOND:On(b, x) ∧ Clear (b),

  EFFECT:On(b, Table) ∧ Clear (x) ∧ ¬On(b, x)) .
- Second, we take the interpretation of Clear (x) to be "there is a clear space on x to hold a block." Under this interpretation, Clear (Table) will always be true.
- The only problem is that nothing prevents the planner from using Move (b, x, Table) instead of MoveToTable(b, x).
- We could live with this problem—it will lead to a larger-than-necessary search space but will not lead to incorrect answers—or we could introduce the predicate Block and add Block(b)∧ Block(y) to the precondition of Move.

Example: Air cargo transport

$Init(At(C_1, SFO) \land At(C_2, JFK) \land At(P_1, SFO) \land At(P_2, JFK)$
$\land Cargo(C_1) \land Cargo(C_2) \land Plane(P_1) \land Plane(P_2)$
$\land Airport(JFK) \land Airport(SFO))$
$Goal(At(C_1, JFK) \land At(C_2, SFO))$
$Action(Load(c, p, a),$
PRECOND: $At(c, a) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $\neg At(c, a) \land In(c, p))$
$Action(Unload(c, p, a),$
PRECOND: $In(c, p) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $At(c, a) \land \neg In(c, p))$
$Action(Fly(p, from, to),$
PRECOND: $At(p, from) \land Plane(p) \land Airport(from) \land Airport(to)$
EFFECT: $\neg At(p, from) \land At(p, to))$

Figure 10.1 A PDDL description of an air cargo transportation planning problem.

- Figure 10.1 shows an air cargo transport problem involving loading and unloading cargo and flying it from place to place.
- The problem can be defined with three actions: Load, Unload, and Fly.
- The actions affect two predicates: In(c, p) means that cargo c is inside plane p, and At(x, a) means that object x (either plane or cargo) is at airport a. Note that some care must be taken to make sure the At predicates are maintained properly.
- When a plane flies from one airport to another, all the cargo inside the plane goes with it.
- In first-order logic it would be easy to quantify all objects that are inside the plane. But basic PDDL does not have a universal quantifier, so we need a different solution.
- The approach we use is to say that a piece of cargo ceases to be At anywhere when it is In a plane; the cargo only becomes At the new airport when it is unloaded.
- So At really means "available for use at a given location."

- The following plan is a solution to the problem:
- [Load (C1, P1, SFO),Fly(P1, SFO, JFK), Unload(C1, P1, JFK), Load(C2, P2, JFK),Fly(P2, JFK , SFO), Unload(C2, P2, SFO)] .
- Finally, there is the problem of spurious actions such as Fly(P1, JFK, JFK), which should be a no-op, but which has contradictory effects (according to the definition, the effect would include At(P1, JFK) ∧ ¬At(P1, JFK)).
- It is common to ignore such problems, because they seldom cause incorrect plans to be produced. The correct approach is to add inequality preconditions saying that the from and to airports must be different.

Algorithms for Planning with State-Space Search

The most straightforward approach of planning algorithm, is state- space search

- **Forward state-space search (Progression)**
- **Backward state-space search (Regression)**

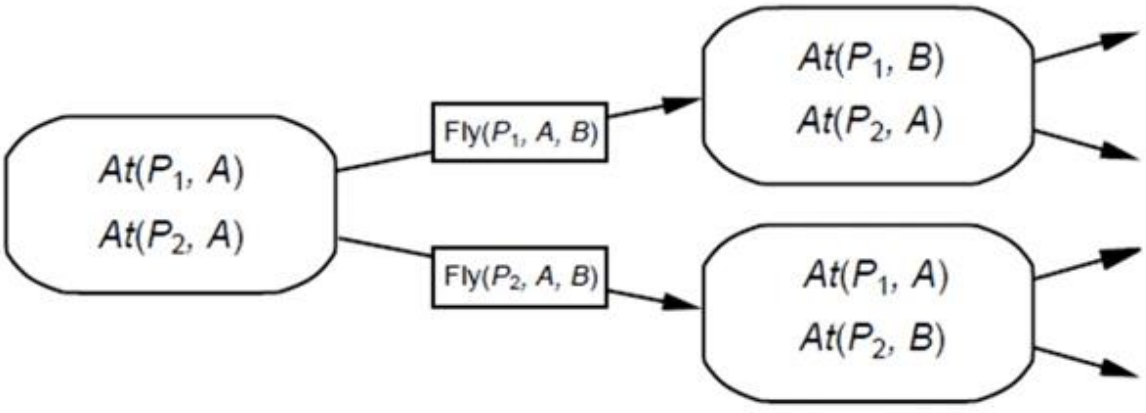
✓ The descriptions of actions in a planning problem and specify both preconditions and effects.

✓ It is possible to search in both directions: either forward from the initial state or backward from the goal.

✓ We can also use explicit action and goal representations, to derive effective heuristics automatically.

Forward State Space Search (Progression):

- From initial state, search forward by selecting operators whose preconditions can be unified with literals in the state
- New state includes positive literals of effect; the negated literals of effect are deleted
- Search forward until goal unifies with resulting state
- This is forward state-space search using STRIPS operators



Problem Formulation for Forward State Space Search (Progression):

Initial state:

- Initial state of the planning problem

Actions:

- Applicable to the current state.
- First actions' preconditions are satisfied, Successor states are generated
- Add positive literals to add list and negative literals to delete list.

Goal test:

- Whether the state satisfies the goal of the planning

Step cost:

- Fach action is 1 (assumed)

FSSS Algorithm

(1) compute whether or not a state is a goal state,

(2) find the set of all actions that are applicable to a state, and

(3) compute a successor state, that is the result of applying an action to a state

- Algorithm takes as input the statement P = (0, s0, g) of a planning problem P. (O contains a list of actions)
- If P is solvable, then Forward-search (O, s0, g) returns a solution plan; otherwise, it returns failure.

0r

1. Forward-search (O, so, g)
2. s←s0
3. л←the empty plan
4. loop
    1. if s satisfies g then return л
    2. applicable {a | a is a ground instance of an operator in O, and precond(a) is true in s}

3. if applicable = Ø then return failure

4. Non-deterministically choose an action a E applicable

5. s←y(s, a)

6. л←л.a

Example: Transportation of air cargo between airports.

Init (At(C1, SFO) ^ At(C2, JFK) ^ At(P1, SFO) ^ At(P2, JFK) ^ Cargo (C1) ^Cargo (C2)

Plane (P1) ^Plane (P2)^ Airport (JFK) ^ Airport (SFO))

Goal (At(C1, JFK) ^ At(C2, SFO))

Action (Load(c, p, a),

PRECOND: At(c, a) ^ At(p, a) ^ Cargo (c) ^ Plane (p) ^ Airport (a)

EFFECT: ¬ At(c, a) ^ In(c, p))

Action (Unload (c, p, a),

PRECOND: In(c, p) ^ At(p, a) ^ Cargo (c) Plane (p) ^ Airport (a)

EFFECT: At(c, a) ^ ¬ In(c, p))

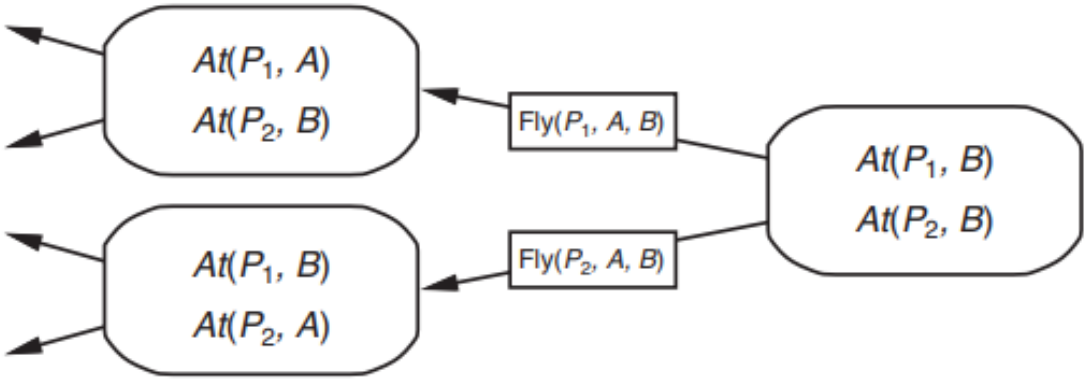
Action (Fly(p, from, to),

PRECOND: At(p, from) ^ Plane (p) ^ Airport (from) ^ Airport (to)

EFFECT: At(p, from) i At(p, to))

Backward state-space search (Regression):

- The goal state must unify with at least one of the positive literals in the operator's effect.
- Its preconditions must hold in the previous situation, and these become subgoals which might be satisfied by the initial conditions.
- Perform backward chaining from goal.
- Again, this is just state-space search using STRIPS operators.



- Backward state search starting from the goal state(s).
- Using the inverse of the actions to search backward for the initial state.

Problem Formulation for Regression:

Initial state:

- Initial state of the planning problem

Actions:

- Applicable to the current state.
- First actions' preconditions are satisfied, Successor states are generated
- Add positive literals to add list and negative literals to delete list.

Goal test:

- Whether the state satisfies the goal of the planning

Step cost:

- Each action is 1 (assumed)

BSSS Algorithm:

- Start at the goal,
- Test the goal is initial state, otherwise
- Apply inverses of the planning operators to produce subgoals,
- The algorithm will stop if we produce a set of subgoals that satisfies the initial state.

Or

1. Backward-search (O, so, g)
2. π ←the empty plan
3. loop
   1. if so satisfies g then return π
   2. applicable {a | a is a ground instance of an operator in O that is relevant for g}
   3. if applicable = Ø then return failure
   4. Non-deterministically choose an action a € applicable
   5. π←α. π
   6. g←y-1(g, a)

PLANNING GRAPHS

All the heuristics we have suggested can suffer from inaccuracies. Here we use a special data structure called a planning graph can be used to give better heuristic estimates.

These heuristics can be applied to any of the search techniques we have seen so far. Alternatively, we can search for a solution over the space formed by the planning graph, using an algorithm called **GRAPHPLAN**.

A planning problem asks if we can reach a goal state from the initial state. Suppose we are given a tree of all possible actions from the initial state to successor states, and their successors, and so on.

If we indexed this tree appropriately, we could answer the planning question "can we reach state G from state S0" immediately, just by looking it up. Of course, the tree is of exponential size, so this approach is impractical.

A planning graph is polynomial size approximation to this tree that can be constructed quickly. The planning graph can't answer definitively whether G is reachable from S0, but it can estimate how many steps it takes to reach G. The estimate is always correct when it reports the goal is not reachable, and it never overestimates the number of steps, so it is an **admissible heuristic.**

A planning graph is a directed graph organized into levels:

first a level S0 for the initial state, consisting of nodes representing each fluent that holds in S0;

then a level A0 consisting of nodes for each ground action that might be applicable in S0; then alternating levels Si followed by Ai; until we reach a termination condition.

Roughly speaking, Si contains all the literals that could hold at time i, depending on the actions executed at preceding time steps.

If it is possible that either P or ¬P could hold, then both will be represented in Si. Also roughly speaking, Ai contains all the actions that could have their preconditions satisfied at time i.

We say "roughly speaking" because the planning graph records only a restricted subset of the possible negative interactions among actions; therefore, a literal might show up at level **Sj** when it could not be true until a later level, if at all. Despite the possible error, the level **j** at which a literal first appears is a good estimate of how difficult it is to achieve the literal from the initial state.

Planning graphs work only for propositional planning problems—ones with no variables. it is straightforward to propositionalize a set of action schemas. Despite the resulting increase in the size of the problem description, planning graphs have proved to be effective tools for solving hard planning problems.

$Init(Have(Cake))$
$Goal(Have(Cake) \wedge Eaten(Cake))$
$Action(Eat(Cake)$
PRECOND: $Have(Cake)$
EFFECT: $\neg Have(Cake) \wedge Eaten(Cake))$
$Action(Bake(Cake)$
PRECOND: $\neg Have(Cake)$
EFFECT: $Have(Cake))$

Figure 10.7 The "have cake and eat cake too" problem.

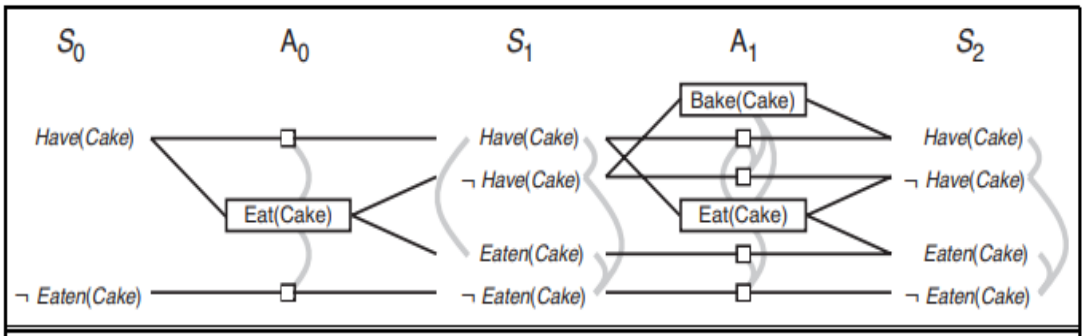


Figure 10.8 The planning graph for the "have cake and eat cake too" problem up to level S_2. Rectangles indicate actions (small squares indicate persistence actions), and straight lines indicate preconditions and effects. Mutex links are shown as curved gray lines. Not all mutex links are shown, because the graph would be too cluttered. In general, if two literals are mutex at S_i, then the persistence actions for those literals will be mutex at A_i and we need not draw that mutex link.

Figure 10.7 shows a simple planning problem, and Figure 10.8 shows its planning graph.

Each action at level **Ai** is connected to its preconditions at **Si** and its effects at **Si+1**. So a literal appears because an action caused it, but we also want to say that a literal can persist if no action negates it.

This is represented by a persistence action (sometimes called PERSISTENCE ACTION a no-op). For every literal **C**, we add to the problem a persistence action with precondition **C** and effect **C**.

Level A0 in Figure 10.8 shows one "real" action, **Eat(Cake)**, along with two persistence actions drawn as small square boxes.

Level A0 contains all the actions that could occur in state S0, but just as important it records conflicts between actions that would prevent them from occurring together.

The gray lines in Figure 10.8 indicate mutual exclusion (or mutex) links. For example, **Eat(Cake)** is mutually exclusive with the persistence of either **Have(Cake)** or **¬Eaten(Cake)**. We shall see shortly how mutex links are computed.

Level **S1** contains all the literals that could result from picking any subset of the actions in **A0**, as well as mutex links (gray lines) indicating literals that could not appear together, regardless of the choice of actions.

For example, **Have(Cake)** and **Eaten(Cake)** are mutex depending on the choice of actions in **A0**, either, but not both, could be the result.

In other words, **S1** represents a belief state: a set of possible states. The members of this set are all subsets of the literals such that there is no mutex link between any members of the subset.

We continue in this way, alternating between state level **Si** and action level **Ai** until we reach a point where two consecutive levels are identical. At this point, we say that the graph has leveled off.

The graph in Figure 10.8 levels off at **S2**. What we end up with is a structure where every **Ai** level contains all the actions that are applicable in **Si**, along with constraints saying that two actions cannot both be executed at the same level.

Every **Si** level contains all the literals that could result from any possible choice of actions in **Ai−1**, along with constraints saying which pairs of literals are not possible.

It is important to note that the process of constructing the planning graph does not require choosing among actions, which would entail combinatorial search. Instead, it just records the impossibility of certain choices using mutex links. We now define mutex links for both actions and literals.

A mutex relation holds between two actions at a given level if any of the following three conditions holds:

• **Inconsistent effects:** one action negates an effect of the other. For example, **Eat(Cake)** and the persistence of **Have(Cake)** have inconsistent effects because they disagree on the effect **Have(Cake).**

• **Interference:** one of the effects of one action is the negation of a precondition of the other. For example **Eat(Cake)** interferes with the persistence of **Have(Cake)** by negating its precondition.

• **Competing needs:** one of the preconditions of one action is mutually exclusive with a precondition of the other. For example, **Bake(Cake)** and **Eat(Cake)** are mutex because they compete on the value of the **Have(Cake)** precondition.

A mutex relation holds between two literals at the same level if one is the negation of the other or if each possible pair of actions that could achieve the two literals is mutually exclusive. This condition is called inconsistent support.

For example, **Have(Cake)** and **Eaten(Cake)** are mutex in **S1** because the only way of achieving **Have(Cake)**, the persistence action, is mutex with the only way of achieving **Eaten(Cake),** namely **Eat(Cake).**

In S2 the two literals are not mutex, because there are new ways of achieving them, such as Bake(Cake) and the persistence of Eaten(Cake), that are not mutex.

A planning graph is polynomial in the size of the planning problem. For a planning problem with **l literals** and **a actions**, each **Si** has no more than **l nodes** and **l^2 mutex links**, and each **Ai** has no more than **a + l nodes** (including the no-ops), **$(a + l)^2$ mutex links**, and **2(al + l) precondition** and **effect links**.

Thus, an entire graph with **n levels** has a size of **$O(n(a + l)^2)$.** The time to build the graph has the same complexity.

Graph plan Algorithm

Graph plan algorithm is used to describe a process of extracting a plan directly from a planning graph instead of graph that produces a heuristic. this algorithm consists of two steps which are one after the other in a loop. Initially, it verifies that all the goal literals exist in the current level and mutex links does not exists in any of the pairs. If this case occurs, then a solution can be obtained within a current graph.

A graph is expanded with the addition of actions and state literals for the current level and next level respectively. This process is continued till a solution is obtained or it is known that a solution cannot be obtained.

```
function GRAPHPLAN(problem) returns solution or failure
  graph ← INITIAL-PLANNING-GRAPH(problem)
  goals ← CONJUNCTS(problem.GOAL)
  nogoods ← an empty hash table
  for tl = 0 to ∞ do
    if goals all non-mutex in S_t of graph then
      solution ← EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph), nogoods)
      if solution ≠ failure then return solution
    if graph and nogoods have both leveled off then return failure
    graph ← EXPAND-GRAPH(graph, problem)
```

Figure 10.9 The GRAPHPLAN algorithm. GRAPHPLAN calls EXPAND-GRAPH to add a level until either a solution is found by EXTRACT-SOLUTION, or no solution is possible.

Example:

The operation or working of the graph plan is explained using a spare tire problem. The planning graph for a spare tire problem is shown in figure 10.10 as follows,

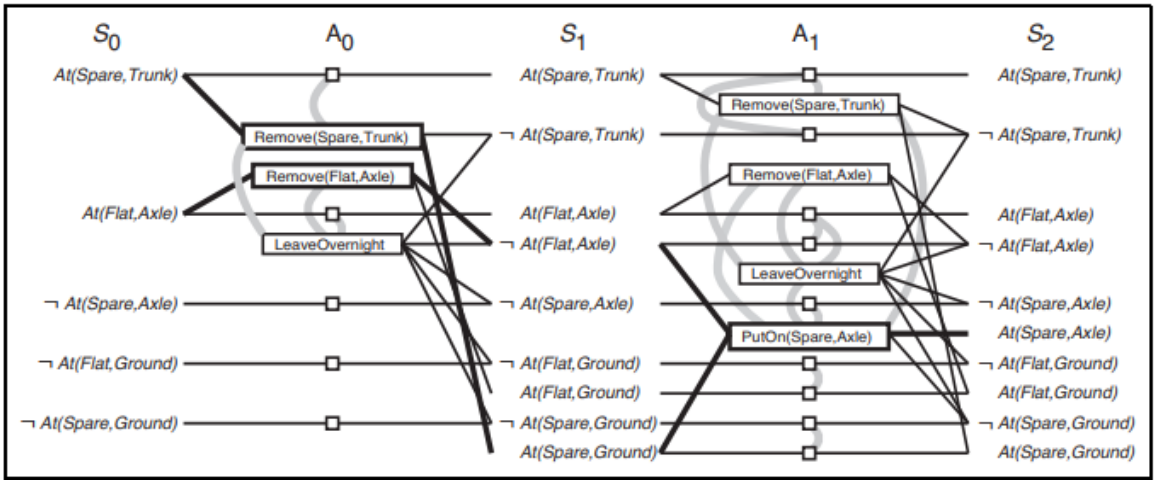


Figure 10.10 The planning graph for the spare tire problem after expansion to level S_2. Mutex links are shown as gray lines. Not all links are shown, because the graph would be too cluttered if we showed them all. The solution is indicated by bold lines and outlines.

The graph is shown in Figure 10.10. The first line of GRAPHPLAN initializes the planning graph to a one-level (S0) graph representing the initial state. The positive fluents from the problem description's initial state are shown, as are the relevant negative fluents. Not shown are the unchanging positive literals (such as Tire(Spare)) and the irrelevant negative literals.

The goal At(Spare, Axle) is not present in S0, so we need not call EXTRACT-SOLUTION— we are certain that there is no solution yet. Instead, EXPAND-GRAPH adds into A0 the three actions whose preconditions exist at level S0 (i.e., all the actions except PutOn(Spare, Axle)), along with persistence actions for all the literals in S0. The effects of the actions are added at level S1. EXPAND-GRAPH then looks for mutex relations and adds them to the graph.

At(Spare, Axle)is still not present in S1, so again we do not call EXTRACT-SOLUTION. We call EXPAND-GRAPH again, adding A1 and S1 and giving us the planning graph shown in Figure 10.10. Now that we have the full complement of actions, it is worthwhile to look at some of the examples of mutex relations and their causes:

- **Inconsistent effects:** Remove(Spare, Trunk) is mutex with LeaveOvernight because one has the effect At(Spare, Ground) and the other has its negation.
- **Interference:** Remove(Flat, Axle) is mutex with LeaveOvernight because one has the precondition At(Flat, Axle) and the other has its negation as an effect.
- **Competing needs:** PutOn(Spare, Axle) is mutex with Remove(Flat, Axle) because one has At(Flat, Axle) as a precondition and the other has its negation.
- **Inconsistent support:** At(Spare, Axle) is mutex with At(Flat, Axle) in S2 because the only way of achieving At(Spare, Axle) is by PutOn(Spare, Axle), and that is mutex with the persistence action that is the only way of achieving At(Flat, Axle). Thus, the mutex relations detect the immediate conflict that arises from trying to put two objects in the same place at the same time.

This time, when we go back to the start of the loop, all the literals from the goal are present in S2, and none of them is mutex with any other. That means that a solution might exist, and EXTRACT-SOLUTION will try to find it.

We can formulate EXTRACT-SOLUTION as a Boolean constraint satisfaction problem (CSP) where the variables are the actions at each level, the values for each variable are in or out of the plan, and the constraints are the mutexes and the need to satisfy each goal and precondition.

Alternatively, we can define EXTRACT-SOLUTION as a backward search problem, where each state in the search contains a pointer to a level in the planning graph and a set of unsatisfied goals. We define this search problem as follows:

- The initial state is the last level of the planning graph, Sn, along with the set of goals from the planning problem.

- The actions available in a state at level Si are to select any conflict-free subset of the actions in Ai−1 whose effects cover the goals in the state. The resulting state has level Si−1 and has as its set of goals the preconditions for the selected set of actions. By "conflict free," we mean a set of actions such that no two of them are mutex and no two of their preconditions are mutex.
- The goal is to reach a state at level S0 such that all the goals are satisfied.
- The cost of each action is 1.

For this particular problem, we start at S2 with the goal At(Spare, Axle).

The only choice we have for achieving the goal set is PutOn(Spare, Axle). That brings us to a search state at S1 with goals At(Spare, Ground) and ¬At(Flat, Axle).

The former can be achieved only by Remove(Spare, Trunk), and the latter by either Remove(Flat, Axle) or LeaveOvernight. But LeaveOvernight is mutex with Remove(Spare, Trunk), so the only solution is to choose Remove(Spare, Trunk) and Remove(Flat, Axle).

That brings us to a search state at S0 with the goals At(Spare, Trunk) and At(Flat, Axle). Both of these are present in the state, so we have a solution: the actions Remove(Spare, Trunk) and Remove(Flat, Axle) in level A0, followed by PutOn(Spare, Axle) in A1.